

DJANGO REST FRAMEWORK — COMPLETE ROADMAP

Beginner → Intermediate → Advanced → Real-Time Backend Developer

Goal: Learn DRF from REST fundamentals through serializers, CRUD, authentication, JWT, permissions, advanced APIs, testing, performance and deployment.

LEVEL 1 — REST API FUNDAMENTALS

1. *What is an API?*

Understand frontend/mobile communication with backend through HTTP requests and JSON responses.

2. *What is REST?*

Learn client-server, stateless communication, resource-based URLs, HTTP methods and JSON.

LEVEL 2 — HTTP & REST METHODS

3. *HTTP Methods*

GET = read, POST = create, PUT = full update, PATCH = partial update, DELETE = remove.

4. *HTTP Status Codes*

200, 201, 204, 400, 401, 403, 404, 405 and 500.

LEVEL 3 — DRF SETUP

5. *Installation*

Install `django-rest-framework`, add `rest_framework` to `INSTALLED_APPS`, create an API app and run the server.

6. *First API*

Create an endpoint using `@api_view` and `Response`. Understand URL → View → Response → JSON.

LEVEL 4 — SERIALIZERS

7. *Serializers*

Convert Django/Python objects to JSON-friendly data and validate incoming data.

8. *ModelSerializer*

Connect serializers with models using `Meta`, `model`, `fields`, `exclude`, `read-only` and `write-only` fields.

9. *Validation*

Learn `validate_()`, `validate()`, `ValidationError`, `required fields` and custom business rules.

LEVEL 5 — API VIEWS

10. *Function-Based API Views*

Build GET, POST, PUT, PATCH and DELETE CRUD APIs.

11. *APIView*

Use class-based API methods: `get()`, `post()`, `put()`, `patch()` and `delete()`.

12. *Generic Views*

Learn `ListAPIView`, `CreateAPIView`, `RetrieveAPIView`, `UpdateAPIView`, `DestroyAPIView` and combined views.

13. *ViewSet*

Use `ModelViewSet` for `list`, `create`, `retrieve`, `update`, `partial_update` and `destroy`.

14. *Routers*

Use DefaultRouter/SimpleRouter to generate URLs for ViewSets.

LEVEL 6 — QUERYING

15. Request & Response

Use request.data, request.query_params, request.user, request.method and Response with status codes.

16. Query Parameters

Read values such as ?name=Kesava using request.query_params.

17. Filtering

Use django-filter and lookup expressions such as age__gte, age__lte and name__icontains.

18. Search

Implement search with search_fields and query parameters.

19. Ordering

Support ascending/descending ordering and ordering_fields.

20. Pagination

Learn PageNumberPagination, LimitOffsetPagination and CursorPagination.

LEVEL 7 — AUTHENTICATION

21. Authentication

Learn Session, Basic, Token and JWT authentication concepts.

22. JWT Authentication

Understand access tokens, refresh tokens, login APIs, protected endpoints and token refresh.

LEVEL 8 — PERMISSIONS & USERS

23. Permissions

Learn AllowAny, IsAuthenticated, IsAdminUser and permission classes.

24. Custom Permissions

Build role-based access for Admin, Teacher and Student.

25. Custom User Model

Learn User, AbstractUser, AbstractBaseUser, custom fields, profiles and roles.

LEVEL 9 — RELATIONSHIPS

26. Relationships in APIs

Expose ForeignKey, OneToOneField and ManyToManyField relationships.

27. Nested Serializers

Return related objects and understand nested read/write data.

LEVEL 10 — FILES, CORS & EMAIL

28. File Upload API

Handle FileField, ImageField, multipart/form-data, request.FILES and media.

29. CORS

Understand cross-origin requests and allowed origins for separate frontend/backend apps.

30. Email / OTP

Build email verification, password reset, OTP and notification workflows.

LEVEL 11 — ADVANCED API FEATURES

31. Error Handling

Create consistent validation, 404, authentication, permission and exception responses.

32. API Documentation

Learn Swagger/OpenAPI and ReDoc; document endpoints, parameters, auth and responses.

33. API Versioning

Use versioned endpoints such as /api/v1/ and /api/v2/.

34. Throttling

Use rate limits to protect APIs from excessive requests.

LEVEL 12 — SECURITY

35. API Security

Authentication, authorization, CSRF, CORS, XSS, SQL injection protection, password hashing, JWT security, secrets, HTTPS and rate limiting.

LEVEL 13 — TESTING

36. Postman

Test GET, POST, PUT, PATCH, DELETE, authentication, permissions, validation and status codes.

37. Automated Testing

Use APITestCase and APIClient to test CRUD, authentication, permissions and errors.

LEVEL 14 — PERFORMANCE & BACKGROUND TASKS

38. Performance

Learn select_related(), prefetch_related(), indexes, N+1 avoidance, pagination and efficient serializers.

39. Caching

Understand caching and Redis for reducing repeated database work.

40. Background Tasks

Learn Celery and Redis/RabbitMQ concepts for emails, notifications, reports and long-running jobs.

LEVEL 15 — DEPLOYMENT

41. Production Deployment

Learn Git/GitHub, environment variables, DEBUG=False, PostgreSQL, static/media files, Gunicorn, Nginx, HTTPS, domains and logs.

LEVEL 16 — REAL-TIME PROJECTS

42. Student CRUD API

Create/read/update/delete students, search and pagination.

43. Authentication API

Registration, login, JWT, refresh token, profile, logout and permissions.

44. E-Commerce REST API

Users, products, categories, cart, wishlist, orders, payments, reviews, JWT, permissions, search, filtering and pagination.

45. College Management API

Admin, teachers, students, departments, courses, attendance, exams, results, notifications, documents, JWT, role-based permissions, Swagger, testing and deployment.

RECOMMENDED TEACHING ORDER

REST → HTTP → API → DRF Setup → First API → Serializers → ModelSerializer → Validation → CRUD → APIView → Generic Views → ViewSets → Routers → Filtering → Search → Ordering → Pagination → Authentication → JWT → Permissions → Custom User → Relationships → Nested Serializers → File Upload → CORS → Email/OTP → Error Handling → Swagger → Versioning → Throttling → Security → Testing → Performance → Redis → Celery → Deployment → Real-Time Project

CORE SKILLS TO MASTER

Serializers → ModelSerializer → CRUD → APIView → Generic Views → ViewSets → Routers → JWT → Permissions → Filtering → Pagination → Testing → Documentation → Deployment